ITERATION LAB

DESIGN · SIMULATE · MACHINE

RESEARCH PAPER · 2026

COMPANION STUDY

THE RESEARCH QUESTION

The Program Was Perfect. The Machine Moved Anyway.

How coordinate conventions, work offsets, tool compensation and setup errors produce unexpected tool motion in a syntactically correct program



MD Ikramul Hoque (Arik)

Researcher · Maker · CNC Programmer

BKTTC SICIP CNC Trainee · Chattogram, Bangladesh

The Iteration Lab · Design · Simulate · Machine

Contents

Abstract	2
Key findings at a glance	2
1. Introduction	3
2. Background: Where the Tool Position Really Comes From	3
3. Method	4
4. Results	5
5. Discussion: Why a Correct Program Moves Wrongly	9
6. Worked Example, Step by Step	10
7. How to Prevent It	11
8. The Software: Phantom Motion Simulator	11
9. Limitations, Future Work and Conclusion	12
References	12
About the Author	13
How to Cite This Paper	13

Key findings at a glance

576 modal states of one single G-code line	**1 of 576** states puts the tool where the programmer intended (0.17%)	**70.7%** of all wrong states move the tool with no alarm
20.2% → 0.0% typical-shop vs. disciplined-shop failure rate	**76.5%** parts failing when one offset is typed 0.05 mm wrong	**92%** of position variance explained by touch-off + vise angle

Abstract

A CNC controller checks a program one line at a time. A part is judged one position at a time. But the tool does not move to “a line”; it moves to a position built from many quantities added together: the active unit, the distance mode, the work offset, the tool-length value, the cutter-radius value, the angle of the vise and the temperature of the spindle. A single wrong quantity in that sum produces wrong motion, and the code that caused it is still perfectly valid.

This paper studies that gap in three layers. A model machine is used to enumerate every modal combination of one ordinary line of G-code: 576 states, of which exactly one is correct. A closed-form analysis shows how fast small setup errors (angle, radius, offset) grow. A one-million-part Monte Carlo simulation per scenario shows what these errors do to a real part when they stack, echoing the stack-up study that preceded this one.

The result: in a model of a typical manual shop, about one part in five is out of tolerance although every program line is syntactically correct and every individual setting is “reasonable”. With a single work offset typed 0.05 mm wrong, the figure becomes three parts in four. The fix that matters is not a better G-code editor but a better measurement of the setup: probing the offsets and the work angle alone cuts the failure rate from 20.2% to 2.8%.

Core idea: *G-code is checked for grammar. The part is made by geometry. The two do not talk to each other.*

Keywords: CNC programming, G-code, work offset, tool length compensation, cutter radius compensation, modal state, setup error, Monte Carlo, process drift

1. Introduction

Every CNC trainee meets the same moment. The program is typed, the editor shows no error, the controller accepts it, the graphic simulation looks fine. Then the cycle starts and the tool goes somewhere nobody expected. Sometimes it is a quiet 0.08 mm too far. Sometimes it is a loud 75 mm too deep.



A real finished feature: a threaded stub turned on a manual lathe. Every pass measured correctly on its own — the question this paper asks is what else was added before the cut, unseen on the program sheet.

In both cases the program was syntactically correct. Nothing was misspelled. The failure lives somewhere else, in the quantities that are **not written on the line you are reading**:

- the unit that was switched on three programs ago (G20 or G21),
- the distance mode left active (G90 or G91),
- the work offset register (G54 … G59) and the numbers stored in it,
- the tool-length register (H) and the radius register (D),
- the side of the path the tool is offset to (G41 / G42),
- the angle at which the workpiece sits in the vise,
- the temperature of the machine on that particular morning.

This paper calls these **hidden operands**. A G-code line has visible operands (X60. Y40.) and hidden ones. The position of the tool is the sum of both.

The previous study in this series showed that parts are inspected one at a time but fail together. This study shows its twin: **lines are checked one at a time, but they crash together.**

1.1 Research question

“How can coordinate conventions, work offsets, tool compensation and setup errors produce unexpected tool motion even when a program appears syntactically correct, and how large are these effects?”

1.2 Contributions

- A position equation that separates visible from hidden operands (Section 2).
- A complete enumeration of 576 modal states for one line of code and the share that fail silently (Section 4.1).

- A single-fault catalogue with the size of each error in millimetres (Section 4.2).
- Closed-form growth laws for angular and radius errors (Sections 4.3 and 4.4).
- A Monte Carlo study of seven shop scenarios, with a variance breakdown and a mitigation ladder (Sections 4.5 to 4.8).
- An open, reproducible program, phantom_motion_sim.py (Section 8).

2. Background: Where the Tool Position Really Comes From

2.1 The position equation

For a point programmed as **p** = (X, Y, Z) in work coordinates, the position the tool tip actually reaches in machine coordinates is:

$$P_{machine} = O_{wcs} + R(\theta)\cdot(k\cdot p) + c_{comp} + \delta_{thermal}$$

Symbol	Meaning	Where it comes from
p	programmed coordinate	**visible** — on the line
k	unit factor (1 for mm, 25.4 for inch) and decimal-point scaling	**hidden** (G20/G21, controller parameters)
(distance mode)	absolute G90 or incremental G91	**hidden** (modal)
O_{wcs}	work offset G54…G59	**hidden** (register values)
R(θ)	rotation of the workpiece relative to the axes	**hidden** (setup)
c_{comp}	cutter-radius and tool-length compensation	**hidden** (G41/G42/G43, registers)
$\delta_{thermal}$	thermal growth	**hidden** (environment)

Only the first term is on the line. Everything else is state.

2.2 Four families of hidden error

Family	Typical cause	Typical size	Who notices
Convention	wrong unit, wrong distance mode, missing decimal point, wrong plane	mm to metres	often nobody, until the cut
Offset	wrong work register, typo in register, wrong sign	0.01 mm to hundreds of mm	first-article inspection
Compensation	wrong H or D, cancelled or wrong-side comp, worn tool	0.01 mm to 75 mm	sometimes the operator, sometimes the spindle
Setup	vise skew, part not seated, thermal growth	0.01 to 0.5 mm	the inspection report

2.3 Why this resembles the tolerance stack

In the earlier paper the closing dimension was C = S − (P1 + … + P5). Here the *closing position* is the sum above. Each hidden operand is a link in the chain. Each one is “allowed to wander a little”. And the wandering accumulates, with the extra twist that most of these links share a **common cause** (the same operator, the same touch-off, the same warm-up) — precisely the correlated case the earlier paper found to be dangerous.

3. Method

The study has three layers so each result can be cross-checked against the others.

Layer 1: Enumeration.

A model machine (travel X 0 to −700, Y 0 to −450, Z 0 to −400 mm; six work offsets; a 75.000 mm tool with a 5 mm radius) executes the single line `G01 X60. Y40.` from the point (10, 10). Every combination of the following is evaluated:

Hidden state	Options	Count
Unit	G20, G21	2
Distance mode	G90, G91	2
Work offset	G54, G55, G56, G57, G58, G59	6
Cutter comp	G40, G41, G42	3
Length comp	H1 (correct, 75.0), H2 (82.4), H3 (68.6), none	4
Decimal point	present, missing	2
Total states	2 × 2 × 6 × 3 × 4 × 2	**576**

The intended state is G21 G90 G54 G41 H1 with the decimal point present. A state is **wrong** if the tool tip or cutter centre ends 0.05 mm or more from the intended location. A wrong state is **loud** if it drives an axis outside the stroke limits (the controller alarms) and **silent** otherwise.

Layer 2: Closed-form analysis.

Angular error $e = L \cdot \sin\theta$ and radius-compensation error $\Delta W = 2 \cdot \Delta r$ are evaluated over the practical range.

Layer 3: Monte Carlo.

A test part has three features, each with a tolerance of ±0.05 mm: a hole located at X = 80, Y = 50 mm from the datum (X and Y checked separately), a pocket width of 20 mm cut with cutter-radius compensation, and a depth controlled by tool-length compensation. One million parts are simulated per scenario with these error sources:

Source	Model
Work offset (X and Y)	normal, mean b_o, standard deviation σ_o
Vise rotation	normal, σ_θ degrees; feeds $-Y \cdot \sin\theta + X \cdot (\cos\theta - 1)$ in X
Thermal growth	11.5×10^{-6}/°C × L × ΔT, L=400mm (X), 300mm (Y)
Radius error	normal σ_r plus uniform wear between 0 and w_{max} (one-directional)
Length error	normal σ_h

All sources are independent except that wear and thermal growth are one-directional, which is what makes them dangerous. The seed is fixed (2026), so every number can be reproduced.

Scenarios (σ values in mm, θ in degrees)

Scenario	σ_o	σ_θ	σ_r	w_{max}	σ_h	ΔT mean/sd	Extra
S1 Disciplined (probing, warm-up)	0.005	0.005	0.003	0.005	0.005	1.5/0.5	—
S2 Typical manual shop	0.020	0.015	0.008	0.015	0.015	3/1.5	—
S3 Rushed shift	0.040	0.050	0.015	0.030	0.030	5/2	—
S4 Typical + offset 0.05 off	← as S2 →						b_o=+0.05
S5 Typical + heavy tool wear	← as S2 →						w_{max}=0.06
S6 Typical + cold start	← as S2 →					8/2	—
S7 Typical + vise skew	← as S2 →						θ bias=+0.10°

Important: *the scenario parameters are assumed shop conditions, chosen to be realistic and to span careless-to-careful practice. They are not measurements. The study quantifies how such conditions propagate; it does not claim to measure any particular shop (see Section 9).*

4. Results

4.1 One line of code, 576 meanings

Of the 576 modal states, only 1 reproduces the intended position. The other 575 (99.8%) are wrong by at least 0.05 mm.

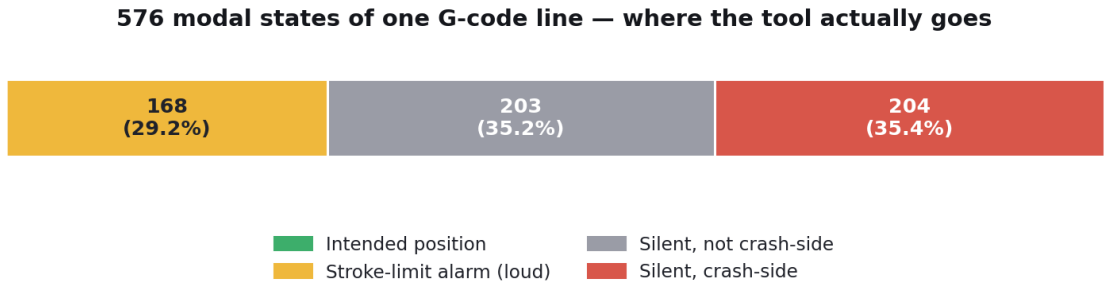


Figure 1. Outcome of every modal state for one ordinary line of G-code.

Outcome	States	Share of all 576	Share of the 575 wrong
Intended position	1	0.2%	n/a
Wrong, loud (stroke-limit alarm)	168	29.2%	29.2%
Wrong, silent (no alarm)	407	70.7%	70.8%
...of which on the crash side (tip lower than intended)	204	35.4%	35.5%

Two conclusions follow. First, the controller’s built-in protection catches fewer than one in three wrong states. Second, roughly half of the silent failures are on the dangerous side (the tool is deeper than intended).

4.2 The single-fault catalogue

Each row changes exactly one hidden operand from the intended state:

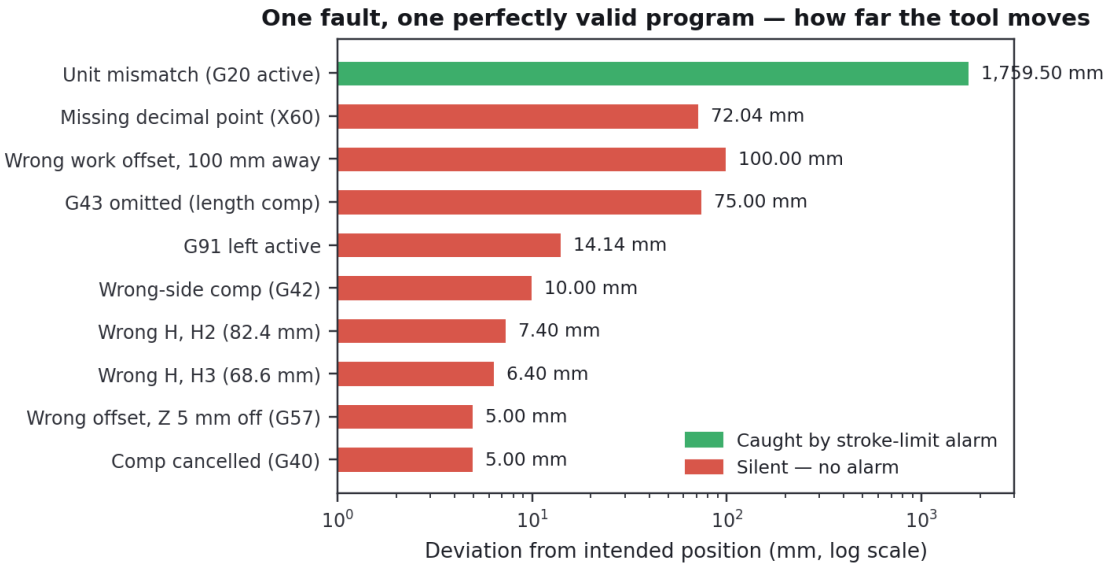


Figure 2. One fault at a time — deviation from the intended position, log scale.

Single fault	Deviation	Vertical part (mm)	Caught by alarm?
Unit mismatch (G20 active, mm program)	1 759.5 mm	0	yes
Missing decimal point (X60 read as 0.060 mm)	72.04 mm	0	no
Wrong work offset, 100 mm away (G55, G56)	100.00 mm	0	no
G43 omitted	75.00 mm	−75.00	no
G91 left active from earlier program	14.14 mm	0	no
Wrong-side cutter comp (G42 instead of G41)	10.00 mm (=2r)	0	no
Wrong H number, H2 (82.4 mm)	7.40 mm	+7.40 (air cut)	no
Wrong H number, H3 (68.6 mm)	6.40 mm	−6.40 (gouge)	no
Wrong work offset, Z 5 mm different (G57)	5.00 mm	−5.00 / +5.00	no
Cutter comp cancelled (G40)	5.00 mm (=r)	0	no

Only 1 of 10 single faults is stopped by the machine itself. Every other one is a valid program doing exactly what it was told, in the wrong place.

A note on the decimal point. *On controllers using standard decimal input and a least input increment of 0.001 mm, X60 without a decimal point is read as 0.060 mm. The shop folklore (“the dot is the most expensive character in G-code”) is, at least in this model, correct to the millimetre: 72.04 mm of deviation from one missing dot.*

4.3 The angular growth law

A workpiece rotated by an angle θ in the vise moves a feature at distance L from the datum sideways by e = L·sin θ.

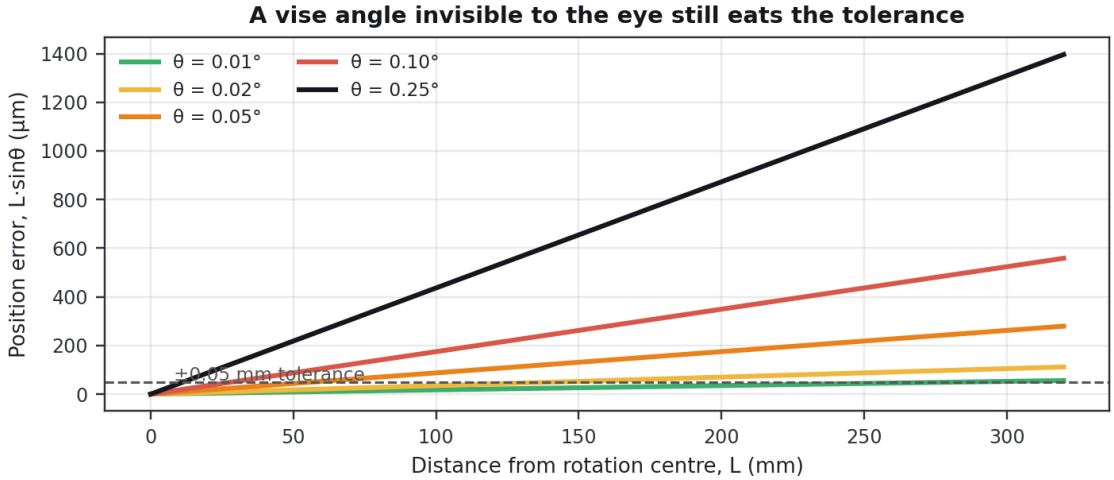


Figure 3. A vise angle invisible to the eye still consumes the tolerance.

Vise angle θ	Error at L=100mm	L=200mm	L=300mm
0.01°	0.017 mm	0.035 mm	0.052 mm
0.02°	0.035 mm	0.070 mm	0.105 mm
0.05°	0.087 mm	0.175 mm	0.262 mm
0.10°	0.175 mm	0.349 mm	0.524 mm
0.25°	0.436 mm	0.873 mm	1.309 mm
0.50°	0.873 mm	1.745 mm	2.618 mm

To stay within ±0.05 mm, the work angle must be better than **0.0286°** at 100 mm, **0.0143°** at 200 mm and **0.0095°** at 300 mm. A tilt invisible to the eye (0.0095° is about 1.7 µm per 10 mm) already consumes the entire tolerance at 300 mm. The error grows linearly with distance, so the bigger the part, the smaller the angle you are allowed.

4.4 The radius-compensation law

When the radius register differs from the real cutter radius by Δr, a pocket or slot cut with comp is wrong in width by 2·Δr.

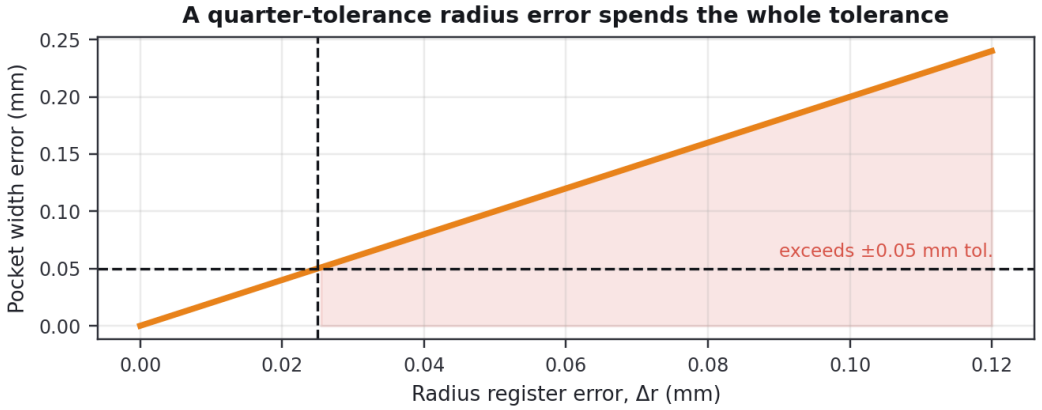


Figure 4. A radius register error is doubled on the part, because it appears on both walls.

Radius error Δr	Width error	Share of a ±0.05 mm tolerance
0.005 mm	0.010 mm	20%
0.010 mm	0.020 mm	40%
0.020 mm	0.040 mm	80%
0.025 mm	0.050 mm	100%
0.050 mm	0.100 mm	200%
0.100 mm	0.200 mm	400%
0.500 mm	1.000 mm	2 000%

The factor of two is the whole story: a radius that is wrong by a quarter of the tolerance already uses all of it, because the error appears on both walls.

4.5 What the Monte Carlo says

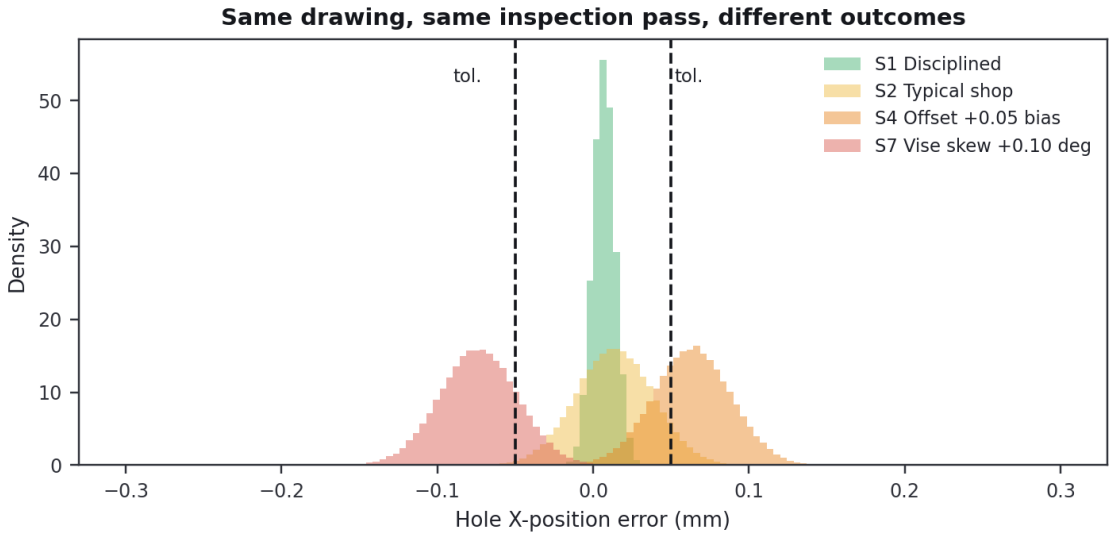


Figure 5. Same drawing, same inspection pass, different outcomes — distribution of the hole X-position error.

Scenario	Any feature failing	Hole position	Pocket width	Depth	Mean X err.	σ of X err.
S1 Disciplined	0.00%	0.00%	0.00%	0.00%	+0.0069	0.0070
S2 Typical manual shop	20.26%	17.96%	2.70%	0.08%	+0.0138	0.0249
S3 Rushed shift	82.37%	72.38%	29.44%	9.58%	+0.0230	0.0599
S4 S2 + offset typed 0.05 off	76.51%	75.84%	2.70%	0.08%	+0.0638	0.0249
S5 S2 + heavy tool wear	65.87%	17.92%	58.41%	0.08%	+0.0138	0.0249
S6 S2 + cold start	50.36%	48.92%	2.72%	0.09%	+0.0368	0.0256
S7 S2 + vise skew 0.10°	99.98%	99.98%	2.69%	0.09%	−0.0736	0.0249

Read the table by comparing S2 with S4 to S7. **The spread of the process (σ of X error) is identical** (0.0249 mm) in S2, S4, S5 and S7. Only the centre moved. That is the same finding as the earlier paper, now in a machining setting: a shifted centre fails parts that a wide but centred process would pass.

4.6 Where the error comes from

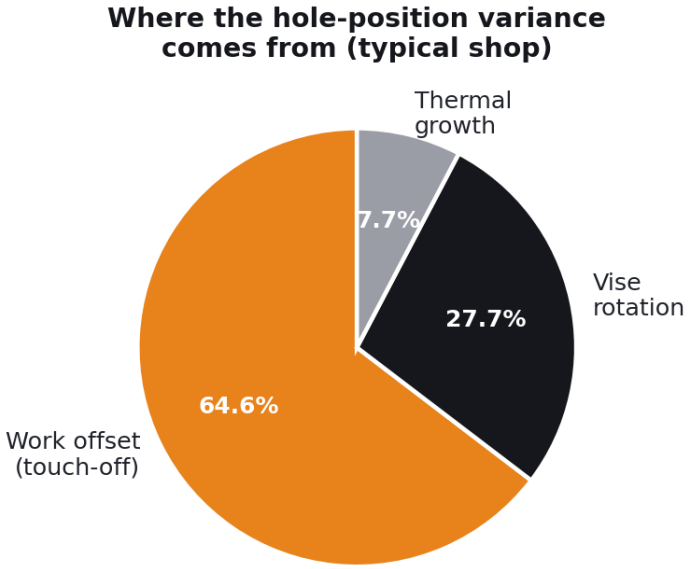


Figure 6. Variance share of the hole-position error in the typical shop (S2).

Source	Share of X-error variance
Work offset (touch-off)	64.7%
Vise rotation	27.7%
Thermal growth	7.7%

Two sources, the touch-off and the vise angle, account for **92%** of the variance. They are both setup quantities, not programming quantities.

RSS versus reality. If the X error is treated as centred and independent (the RSS assumption of the earlier paper), the model predicts σ = 0.0249 mm and a probability of 4.45% that |X error| exceeds 0.05 mm. Applying the same to Y gives roughly 9% for “X or Y”. The Monte Carlo gives 17.96% for the hole, about twice the centred prediction. The gap comes from the thermal and rotation terms, which are not centred (mean X error +0.0138 mm).

4.7 The slow slide, again

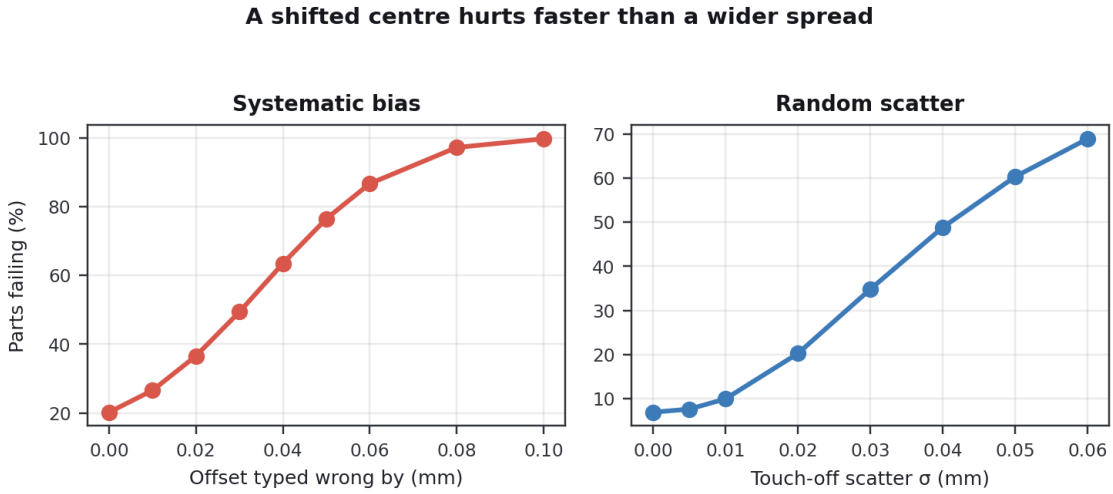


Figure 7. A shifted centre hurts faster than a wider spread of the same size.

Effect of an offset bias (everything else as S2)

Offset wrong by (mm)	0.00	0.01	0.02	0.03	0.04	0.05	0.06	0.08	0.10
Parts failing	20.2%	26.5%	36.5%	49.4%	63.5%	76.4%	86.6%	97.1%	99.7%
Hole position	18.0%	24.4%	34.7%	47.9%	62.5%	75.8%	86.2%	97.1%	99.7%

Effect of touch-off scatter σ (everything else as S2)

σ of touch-off (mm)	0.000	0.005	0.010	0.020	0.030	0.040	0.050	0.060
Parts failing	6.8%	7.6%	9.9%	20.2%	34.8%	48.9%	60.2%	69.0%

Two facts stand out. A systematic offset error (bias) damages the process faster than a random one of the same size: at 0.03 mm, a bias fails 49.4% of parts while a touch-off scatter of σ=0.03 mm fails 34.8%. And even a perfect touch-off (σ=0) still leaves 6.8% failures, because the rest of the setup chain (angle, radius, wear, heat) is not perfect either.

4.8 The mitigation ladder

Starting from the typical shop (S2, 20.2% failing):

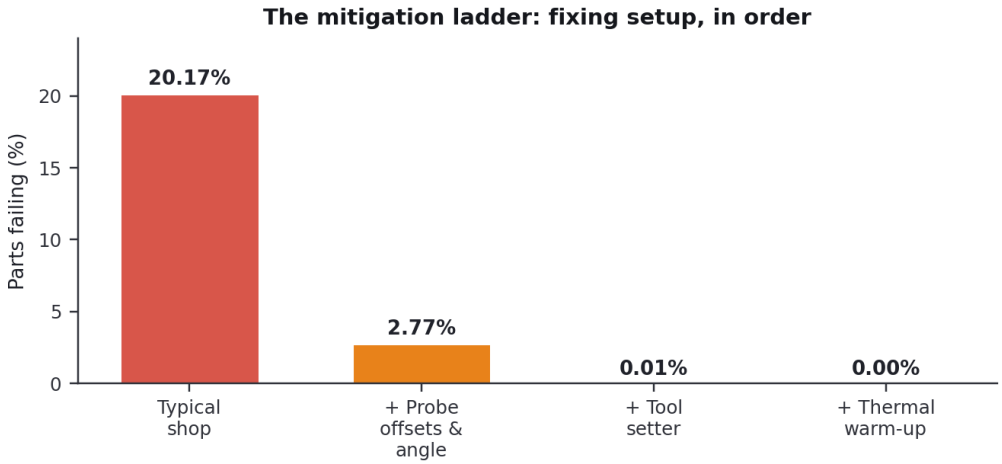


Figure 8. Fixing setup, in order — each bar adds the next intervention.

Intervention	Fix only this one	Cumulative
Probe the offsets and work angle (σ_o 0.005, σ_θ 0.005°)	2.78%	2.77%
Tool setter for radius, length and wear	17.91%	0.01%
Thermal warm-up cycle (ΔT 1.5±0.5 °C)	15.19%	0.00%

Note the shape of the ladder. **Probing alone does 86% of the work** (20.2% → 2.8%). The tool setter alone barely helps (20.2% → 17.9%), because the dominant error is in the offsets, but once the offsets are fixed, the tool setter finishes the job (2.77% → 0.01%). The order of the fixes matters as much as the fixes.

5. Discussion: Why a Correct Program Moves Wrongly

1. The controller checks syntax, not intent. G01 X60. Y40. is valid in 576 modal states. The controller cannot know which one you meant, because the meaning lives in registers and previous lines.

2. The wrong states are mostly quiet. Only 29% of the wrong states trip a stroke-limit alarm. The remaining 71% simply cut, drill or crash, with a clean log.

3. Hidden operands outweigh visible ones. In the single-fault catalogue the visible operand was never at fault. Every deviation, from 5 mm to 1.76 m, came from a quantity that is not on the line.

4. Setup errors dominate. Touch-off and vise angle explain 92% of X-position variance in the typical shop. The G-code can be perfect; the setup then decides.

5. Errors are not centred. Heat, tool wear and a mistyped offset all push in one direction. The statistical comfort of RSS (small errors cancel) is therefore often not available on the shop floor, which is the same warning the previous paper ended with.

6. A tolerance can be consumed before cutting starts. A radius error of 0.025 mm or a vise tilt of 0.0095° at 300 mm uses up a ±0.05 mm tolerance completely. The part is lost in the setup, not in the program.

7. Verification tools check the wrong thing. A graphic simulation draws the path from the code. It shows what the program says, not where the offsets will put it. A program can look perfect on screen and still be 100 mm away on the machine (G55 instead of G54).

6. Worked Example, Step by Step

This section repeats one calculation by hand so the logic is visible without software.

The line: G01 X60. Y40. from (10, 10), with G54 = (−350, −220, −310), tool 75.000 mm, radius 5 mm, comp left (G41).

Step 1: Intended machine position.

X = −350 + 60 − 5 (comp, left of +Y travel) = −295; the centre is offset by r. Y = −220 + 40 = −180. Z (tip at −3) = −310 − 3 = −313.

Step 2: Switch one hidden operand at a time.

Hidden operand changed	New X	New Y	New Z	Deviation
G91 (incremental)	−285	−170	−313	14.14 mm
G55 (Y offset +100)	−295	−80	−313	100.00 mm
G40 (comp off)	−290	−180	−313	5.00 mm
G42 (wrong side)	−285	−180	−313	10.00 mm
H2 = 82.4 (wrong register)	−295	−180	−305.6	7.40 mm (air)
G43 omitted	−295	−180	−388	75.00 mm (crash)

Step 3: Radius error on a real pocket.

Programmed radius 5.000 mm, actual cutter 5.020 mm: Δr = 0.020 mm, width error = 2 × 0.020 = **0.040 mm**, which is already **80%** of a ±0.05 mm tolerance.

Step 4: Angular error.

Vise angle 0.05°, feature at 200 mm: e = 200 × sin(0.05°) = 200 × 0.000873 = **0.175 mm**, which is **3.5 times** the tolerance.

Step 5: Combine them (the stack).

Take the typical-shop offset σ = 0.020 mm, rotation 0.015° at Y = 50 mm (σ ≈ 0.013 mm) and thermal 3 ± 1.5 °C at 400 mm (σ ≈ 0.007 mm):

$$\sigma_X = \sqrt{(0.020^2 + 0.013^2 + 0.007^2)} \approx 0.025 \text{ mm}$$ **(matches the simulation: 0.0249)**

The same drawing that gave 0.00% failures in S1 gives 20.3% here, with exactly the same G-code.

7. How to Prevent It

Strategy	What it does	Evidence
Start every program with a safe line (G21 G90 G40 G49 G80 G17)	Resets the modal state so only 1 of 576 states can occur	Section 4.1
Always write the decimal point	Removes the 72 mm silent error	Section 4.2

Strategy	What it does	Evidence
One work offset per setup, named in the first line	Removes the 100 mm class of errors	Section 4.2
Probe the work offset and angle	The single biggest improvement	20.2% → 2.8% (4.8)
Measure tools with a setter, never by eye	Fixes radius, length and wear	2.77% → 0.01% after probing
Warm up the spindle before cutting	Shrinks the shifted centre	Section 4.8
Track the mean of measured parts, not only pass/fail	Catches drift before it fails	Section 4.5 (S2 vs S4)
Read the offset table aloud before pressing Cycle Start	A human check of the hidden operands	Section 5, point 7
Run the first part single-block with feed override low	Turns a 75 mm crash into a 75 mm look	Section 4.2

In CNC terms: the hidden operands are shared by every part in the batch. A wrong offset is not one bad part; it is all the parts. That is exactly the correlated, drifting case that produced the highest failure rates in this paper.

8. The Software: Phantom Motion Simulator



A real finished thread, held for inspection. A thread form that looks correct on the bench says nothing about which modal state, offset or comp register produced it — that is exactly the blind spot this simulator targets.

8.1 What it is

`phantom_motion_sim.py` (v1.0) is a small Python program written for this study. It answers: if a program is syntactically correct, how often does the tool still go wrong, and how much does the setup contribute? It needs only Python and NumPy.

8.2 How it works

Stage	What happens
1. Enumerate	Builds all 576 modal states for the test line and evaluates the tool position in each
2. Classify	Marks each state as intended, loud (stroke alarm) or silent, and silent states as crash-side or not
3. Closed form	Prints the angular and radius-compensation tables
4. Generate	Draws 1,000,000 virtual parts per scenario with the error sources of Section 3
5. Report	Prints failure rates, variance shares, the mitigation ladder and the sweeps

8.3 How to run

```
python phantom_motion_sim.py
```

The fixed seed (2026) makes every number in this paper repeatable. Monte Carlo values vary by about ±0.05 percentage points between runs with different seeds.

8.4 Limits of the software

It models a three-feature part and a single line of G-code. It ignores deflection, spindle runout, backlash, arc interpolation errors, rotary axes and fixture compliance. Offsets, unit travel limits and

scenario sigmas are stated assumptions. Treat it as a teaching and checking tool, not as a machine-tool simulator.

9. Limitations, Future Work and Conclusion

Limitations

All numbers come from a model. **No physical machine was measured in this version.** The scenario parameters (σ_o, σ_θ and others) are assumptions chosen to be plausible, so the absolute failure percentages (for example 20.3% in S2) describe the model shop, not any real one. The relative findings are the robust part: centre shifts hurt more than scatter, setup dominates program, and probing before tooling is the right order. Real error distributions can be skewed or multi-peaked, and controllers differ in their decimal and modal behaviour.

Your turn to measure (Section 9.1: Field Measurements)

A physical version needs only a dial test indicator and a caliper:

1. Touch off the same corner ten times and record the offset values (gives a real σ_o).
2. Indicate the vise jaw over 100 mm for ten setups (gives a real σ_θ).
3. Measure the same cutter's diameter with a micrometer after one, ten and fifty parts (gives real wear).
4. Cut ten test pockets and compare the real failure rate with the model's prediction.

Conclusion

A G-code line does not describe a position; it describes a request that is completed by hidden state. A program can be flawless in syntax and still place the tool anywhere from 0.05 mm to 1.76 m from where you meant it, and in this model about 70% of those wrong places are reached without a single alarm. In a typical manual-shop model one part in five fails with perfect code, and **setup, not programming, causes 92% of the positional variance**. Measuring the setup (probing) removes most of the failures; the rest of the chain only matters after that.

The lesson: *Don’t only ask “is the program correct?” Ask “what does the machine believe the program means, and what has been added to it?”*

References

1. Fischer, B. R. (2011). *Mechanical Tolerance Stackup and Analysis* (2nd ed.). CRC Press.

2. ISO 6983-1, *Automation systems and integration: Numerical control of machines: Program format and definitions of address words, Part 1: Data format for positioning, line motion and contouring control systems*.

3. ISO 230-3, *Test code for machine tools, Part 3: Determination of thermal effects*.

4. ASME Y14.5, *Dimensioning and Tolerancing*. The American Society of Mechanical Engineers.

5. Controller manufacturer's programming and operation manual (for example, the Fanuc 0i-series operator's manual) for modal G-codes, work offsets, G41/G42 and G43.

6. Hoque, M. I. (Arik). (2026). *Two Perfect Parts That Refuse to Fit Together*. The Iteration Lab, Chattogram, Bangladesh.

About the Author



MD Ikramul Hoque (Arik)

Maker · CNC Programmer · Developer · Math Coach

1st Merit Position, BKTTC SICIP CNC Trainee

Chattogram, Bangladesh

Arik is a maker from Chattogram who learns by building: on a CNC machine, on a circuit board, in a simulator, in a Linux system running inside a phone, and in busy teaching centers. His work covers CNC programming in Fanuc G-code, CAD modeling, web development and cybersecurity, and he coaches students in mathematics.

He started this research series, *Two Perfect Parts That Refuse to Fit Together* and the experiments after it, to turn everyday engineering puzzles into clear, visual and honest studies that anyone can reproduce. He is preparing to study engineering from 2027.

Contact	
Email	mdikramulhoquearik@gmail.com
Portfolio	IkramulHoque-Arik.github.io
Location	Chattogram, Bangladesh
Status	Open to opportunities · Engineering 2027

ITERATION
LAB
DESIGN · SIMULATE · MACHINE

The Iteration Lab
Design · Simulate · Machine
Engineering studies, tools and experiments.

How to cite this paper

Hoque, M. I. (Arik). (2026). The Program Was Perfect. The Machine Moved Anyway: How coordinate conventions, work offsets, tool compensation and setup errors produce unexpected tool motion in syntactically correct CNC programs. The Iteration Lab, Chattogram, Bangladesh.